Taller MQL – Consultas, agregación e índices en MongoDB

Universidad El Bosque · Bases de Datos 2 · Docente: Ing. Christian Felipe Duarte · Noviembre de 2026

Integrantes: David Felipe Bejarano Montes · Samuel Camelo · José Rodríguez

Este documento reúne las capturas y la explicación de los 16 puntos del taller, ejecutados en MongoDB Atlas desde VS Code: consultas con find (Parte A), agregación (Parte B), escritura e índices (Parte C) y modelado con patrones (Parte D). Los conteos cuadran y todas las consultas de negocio de la Parte D usan índice (`IXSCAN`).

El código de los 16 puntos está en el archivo `taller_bejarano_camelo_rodriguez.js`, con un bloque comentado por punto. Las Partes A, B y C trabajan sobre `sample_mflix`; la Parte D crea su propia base, `taller_parteD`.

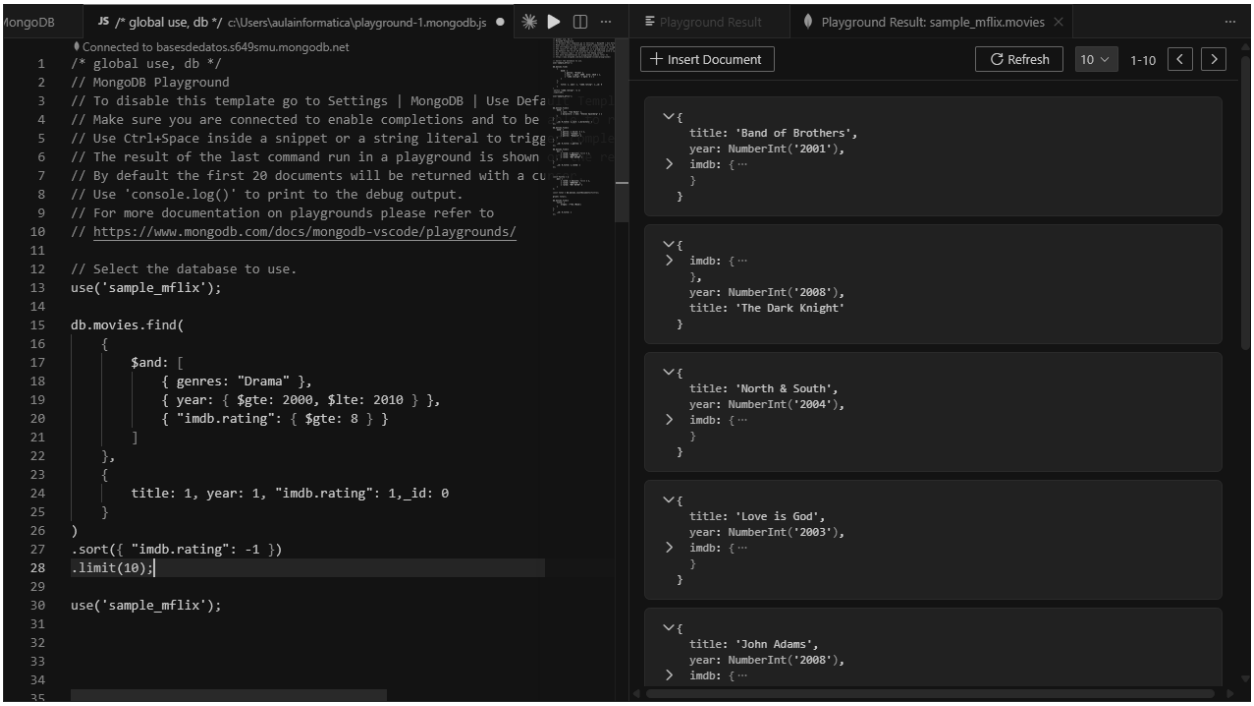
Parte A – Consultas con find

Punto 1 – Drama entre 2000 y 2010 con rating ≥ 8

Consulta:

```
use("sample_mflix");
const filtro1 = { genres: "Drama", year: { $gte: 2000, $lte: 2010 }, "imdb.rating": { $gte: 8 } };
db.movies.find(filtro1, { _id: 0, title: 1, year: 1, "imdb.rating": 1 })
  .sort({ "imdb.rating": -1 })
  .limit(10);
db.movies.countDocuments(filtro1);   // conteo total
```

Resultado:



El filtro combina tres condiciones: el género Drama (MongoDB lo busca dentro del arreglo `genres`), el año entre 2000 y 2010 y `imdb.rating` mayor o igual a 8. Se proyectan solo `title`, `year` e `imdb.rating`, sin `_id`, ordenados por rating descendente y limitados a 10; encabezan *Band of Brothers* y *The Dark Knight*. El conteo total se obtiene con `countDocuments` sobre el mismo filtro, sin el límite.

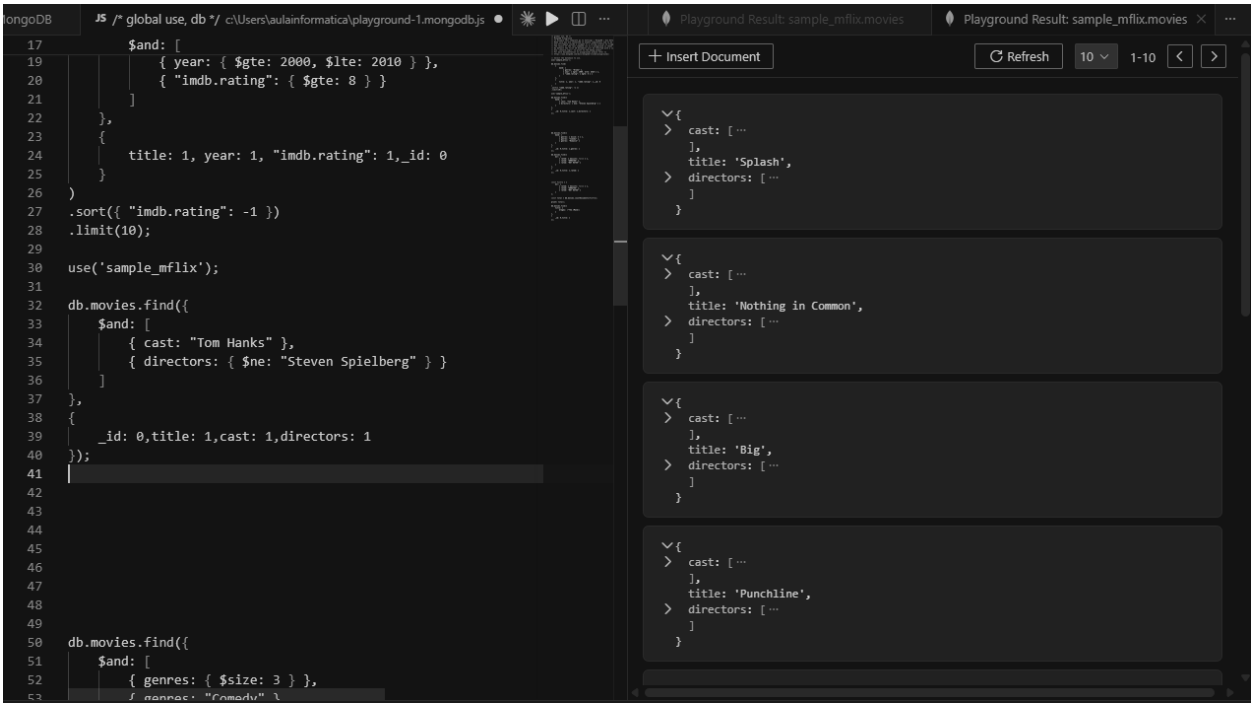
Pendiente: captura del conteo total (línea *Conteo total* del punto 1 al ejecutar el script).

Punto 2 – Películas con Tom Hanks no dirigidas por Steven Spielberg

Consulta:

```
db.movies.find(
  { cast: "Tom Hanks", directors: { $ne: "Steven Spielberg" } },
  { _id: 0, title: 1, year: 1, directors: 1 }
).sort({ year: 1 });
```

Resultado:



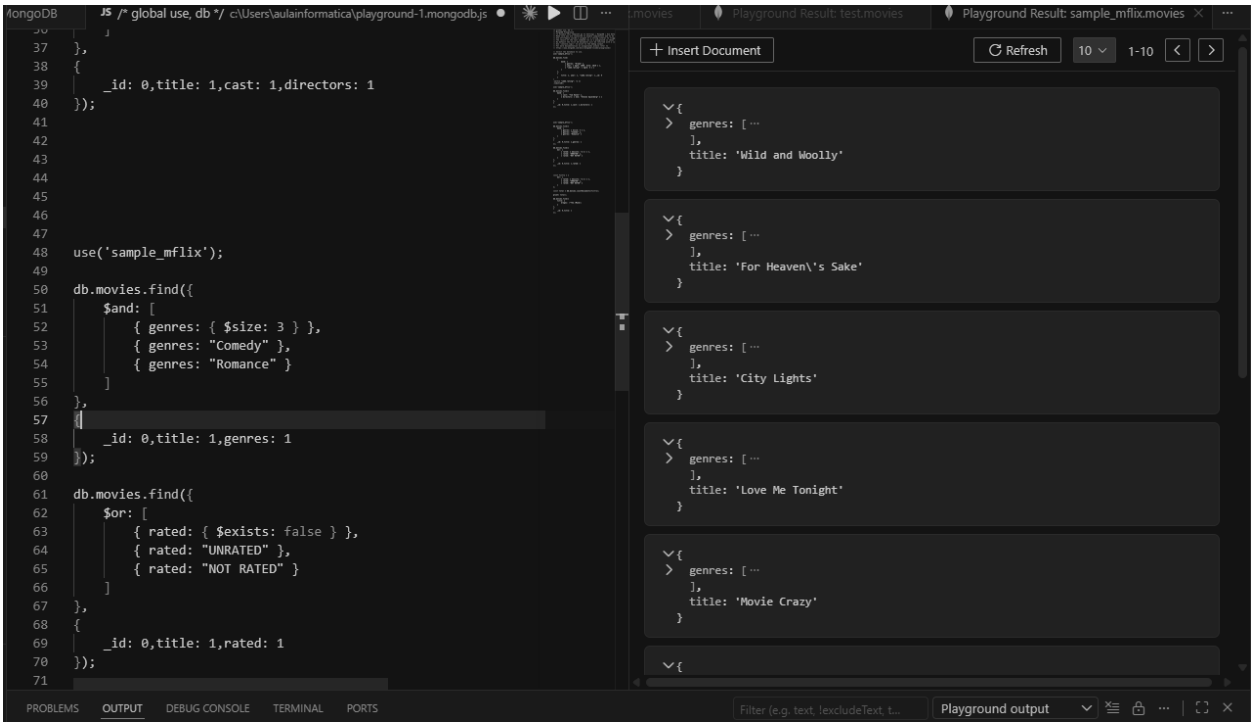
`cast` y `directors` son arreglos: la igualdad con Tom Hanks busca el actor dentro del reparto, y `$ne` sobre `directors` descarta las películas donde cualquiera de los directores sea Spielberg. Aparecen, entre otras, *Splash*, *Nothing in Common*, *Big* y *Punchline*.

Punto 3 – Exactamente 3 géneros, incluidos Comedy y Romance

Consulta:

```
db.movies.find(
  { genres: { $size: 3, $all: ["Comedy", "Romance"] } },
  { _id: 0, title: 1, genres: 1 }
);
```

Resultado:



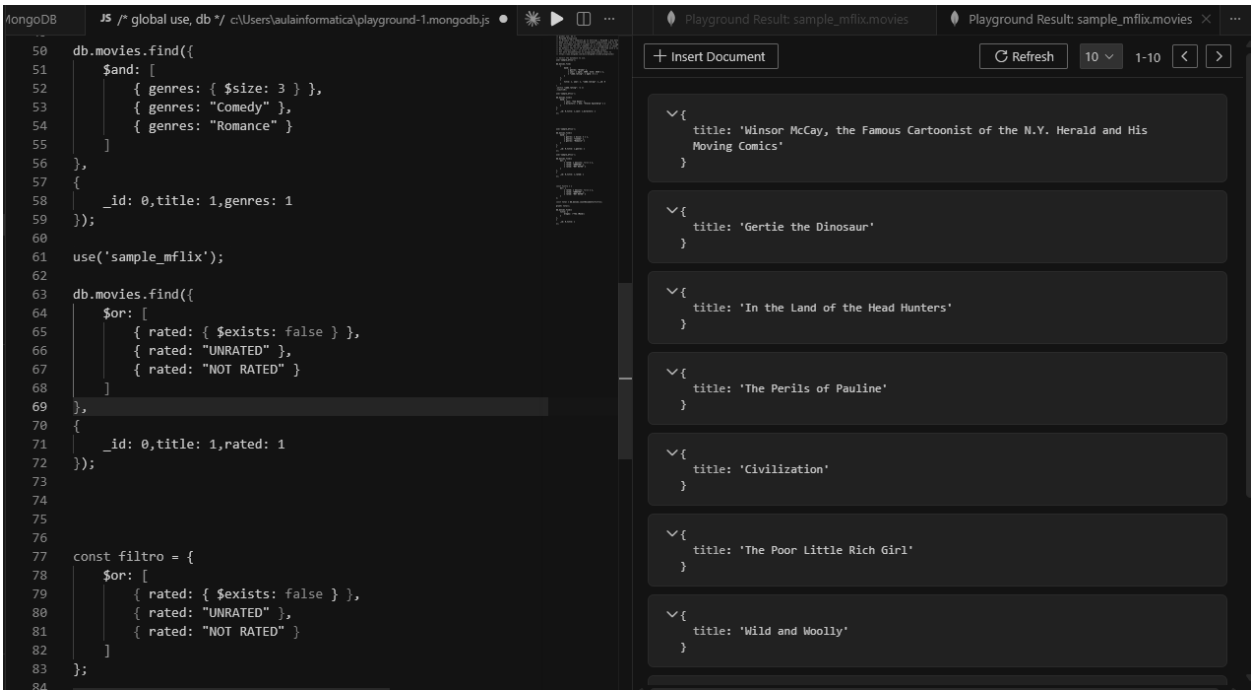
`$size: 3` exige que el arreglo tenga exactamente tres géneros, y las dos condiciones de igualdad exigen que estén Comedy y Romance (equivale a `$all`). La diferencia con `$in` es que este se cumple con uno solo de los valores. Ejemplos: *Wild and Woolly*, *For Heaven's Sake* y *City Lights*.

Punto 4 – Sin clasificación, UNRATED o NOT RATED

Consulta:

```
const filtro4 = { $or: [
  { rated: { $exists: false } },
  { rated: { $in: ["UNRATED", "NOT RATED"] } }
] };
db.movies.find(filtro4, { _id: 0, title: 1, rated: 1 });
db.movies.countDocuments(filtro4);   // conteo
```

Resultado:



`$or` une dos casos: el campo `rated` no existe (`$exists: false`) o su valor es UNRATED o NOT RATED. Las primeras son clásicas del cine mudo, como *Gertie the Dinosaur*, que no tienen clasificación. El conteo se obtiene con `countDocuments` sobre el mismo filtro.

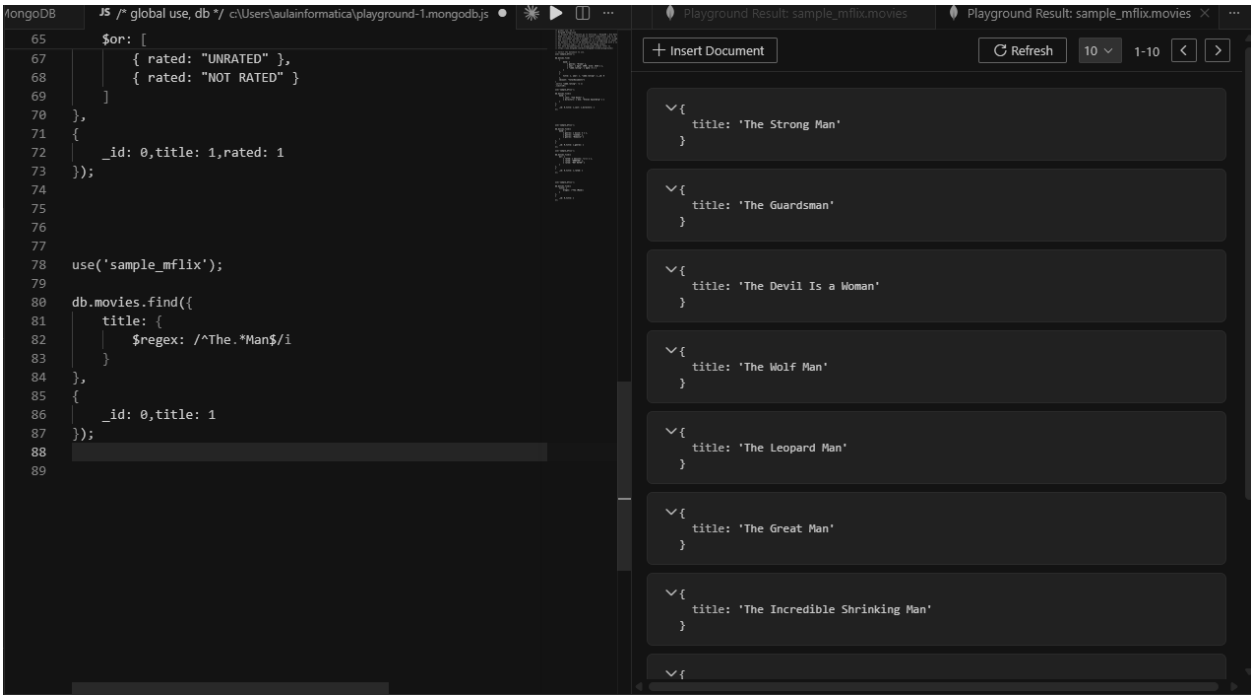
Pendiente: captura del conteo (línea *Conteo* del punto 4 al ejecutar el script).

Punto 5 – Títulos que empiezan por The y terminan en Man

Consulta:

```
db.movies.find(
  { title: { $regex: /^The.*Man$/i } },
  { _id: 0, title: 1 }
).sort({ title: 1 });
```

Resultado:



La expresión regular ancla el inicio con ^The y el final con `Man$`; `.*` permite cualquier texto en medio y la opción `i` ignora mayúsculas. Por eso también aparecen *The Guardsman* y *The Devil Is a Woman*: terminan en “man”. Si solo se quisiera la palabra completa *Man*, se agregaría un límite de palabra antes de *Man*.

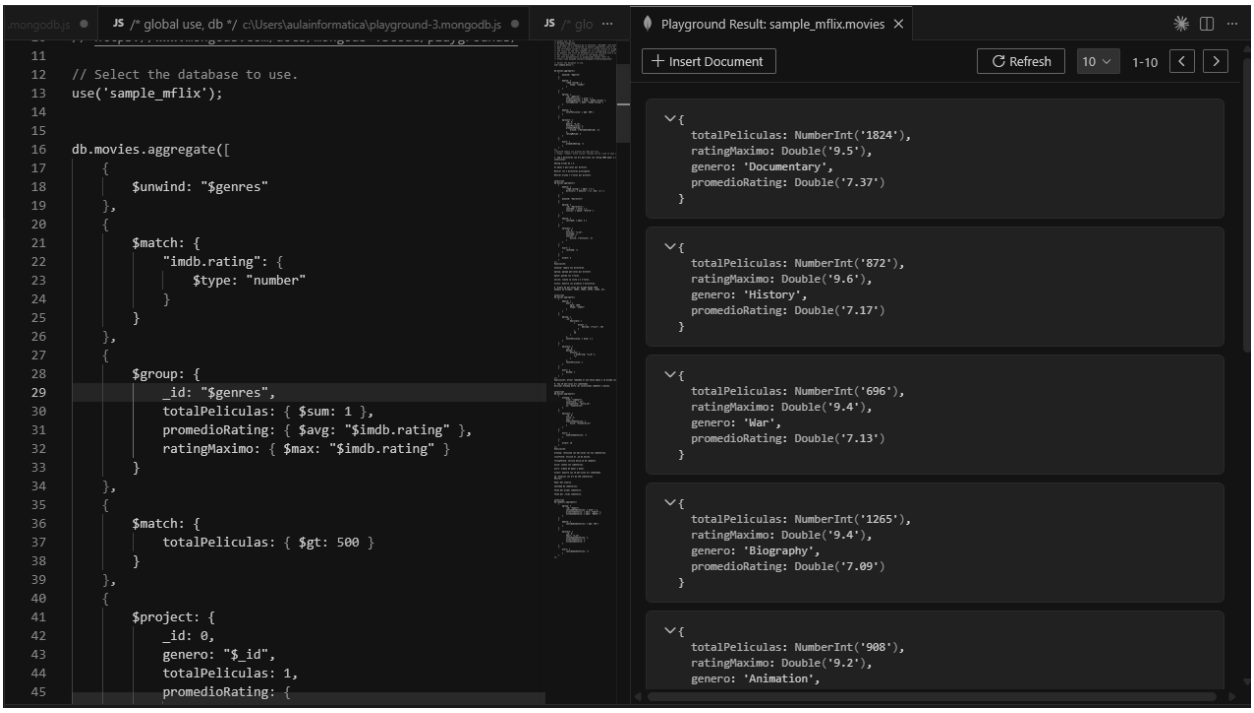
Parte B – Agregación

Punto 6 – Estadísticas por género

Consulta:

```
db.movies.aggregate([
  { $match: { "imdb.rating": { $type: "number" } } },
  { $unwind: "$genres" },
  { $group: { _id: "$genres", totalPeliculas: { $sum: 1 },
              promedioRating: { $avg: "$imdb.rating" }, ratingMaximo: { $max: "$imdb.rating" } } },
  { $match: { totalPeliculas: { $gt: 500 } } },
  { $project: { _id: 0, genero: "$_id", totalPeliculas: 1,
                promedioRating: { $round: ["$promedioRating", 2] }, ratingMaximo: 1 } },
  { $sort: { promedioRating: -1 } }
]);
```

Resultado:



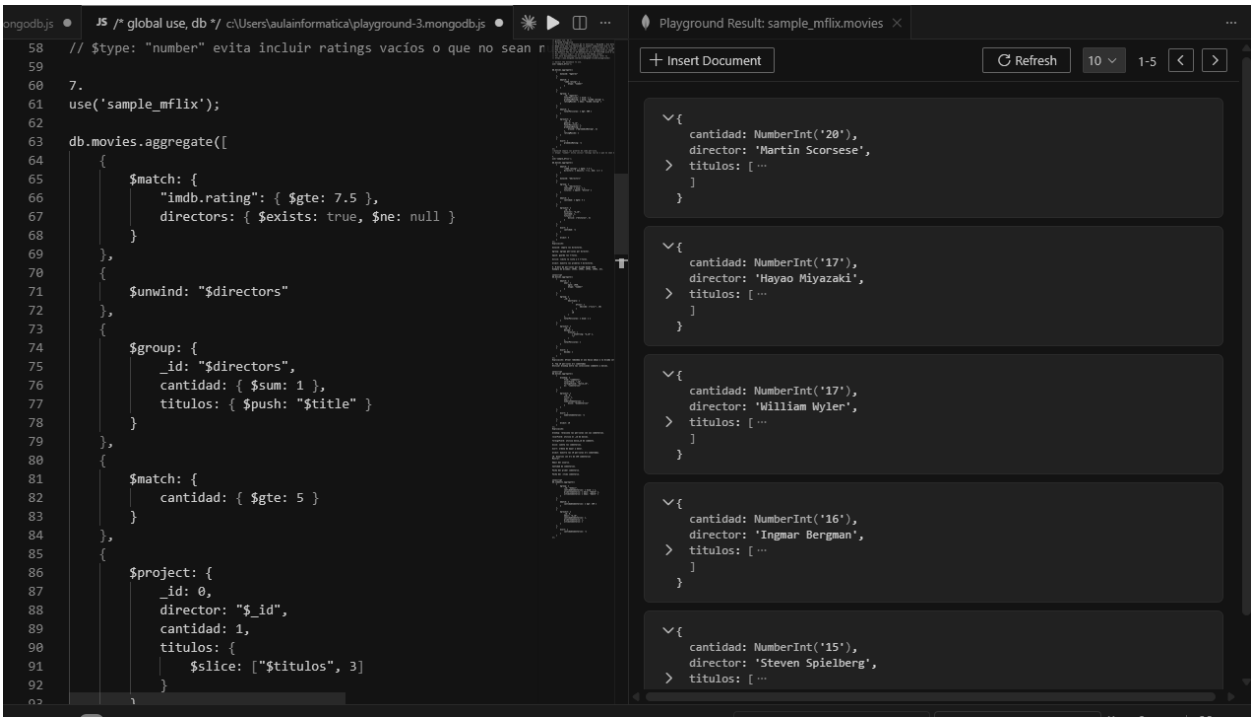
Primero se filtran los ratings numéricos, porque `imdb.rating` tiene valores vacíos; luego `$unwind` crea un documento por género y `$group` calcula cantidad, promedio y máximo. Un segundo `$match` deja los géneros con más de 500 películas, y el promedio se redondea a 2 decimales. Encabeza **Documentary** (1.824 películas, promedio 7,37), seguido de History (7,17) y War (7,13). Filtrar antes de `$unwind` da el mismo resultado pero desenrolla menos documentos.

Punto 7 – Top 5 directores con rating ≥ 7,5

Consulta:

```
db.movies.aggregate([
  { $match: { "imdb.rating": { $gte: 7.5 }, directors: { $exists: true, $ne: null } } },
  { $unwind: "$directors" },
  { $group: { _id: "$directors", cantidad: { $sum: 1 }, titulos: { $push: "$title" } } },
  { $match: { cantidad: { $gte: 5 } } },
  { $project: { _id: 0, director: "$_id", cantidad: 1, titulos: { $slice: ["$titulos", 3] } } },
  { $sort: { cantidad: -1, director: 1 } },
  { $limit: 5 }
]);
```

Resultado:



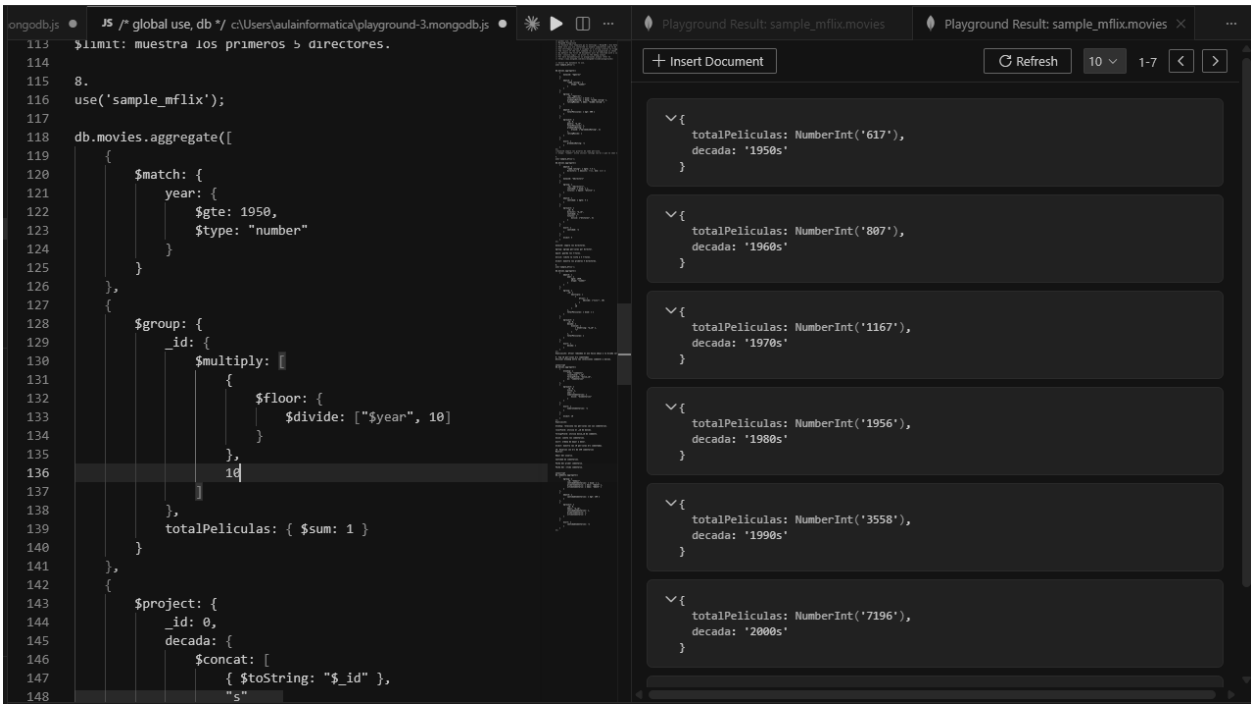
Se filtran las películas con rating de 7,5 o más, `$unwind` separa a los directores, `$group` los cuenta y guarda los títulos con `$push`, y `$slice` deja máximo 3. El top 5 es **Martin Scorsese (20)**, Hayao Miyazaki (17), William Wyler (17), Ingmar Bergman (16) y Steven Spielberg (15).

Punto 8 – Películas por década desde 1950

Consulta:

```
db.movies.aggregate([
  { $match: { year: { $gte: 1950, $type: "number" } } },
  { $group: { _id: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
              totalPeliculas: { $sum: 1 } } },
  { $sort: { _id: 1 } },
  { $project: { _id: 0, decada: { $concat: [{ $toString: "$_id" }, "s"] }, totalPeliculas: 1 } }
]);
```

Resultado:



La década se calcula dividiendo el año entre 10, redondeando hacia abajo con `$floor` y multiplicando por 10; `$concat` le agrega la "s". Se filtran solo los años numéricos, porque `year` tiene algunos valores de texto. La producción crece cada década: de 617 películas en los 1950s a **7.196 en los 2000s**.

Punto 9 – Top 10 películas más comentadas

Consulta:

```
db.comments.aggregate([
  { $group: { _id: "$movie_id", numeroComentarios: { $sum: 1 } } },
  { $sort: { numeroComentarios: -1 } },
  { $limit: 10 },
  { $lookup: { from: "movies", localField: "_id", foreignField: "_id", as: "pelicula" } },
  { $unwind: "$pelicula" },
  { $project: { _id: 0, titulo: "$pelicula.title", anio: "$pelicula.year", numeroComentarios: 1 } }
]);
```

Resultado:

Pendiente: la captura que había en el Word mostraba el resultado del punto 10, no el de este punto. Falta la captura del top 10 al ejecutar el script.

Se agrupan los comentarios por `movie_id`, se ordenan de mayor a menor y se dejan los 10 primeros; después `$lookup` trae de `movies` el título y el año. Hacer el `$lookup` al final lo ejecuta 10 veces en lugar de 21.349, y el resultado es el mismo.

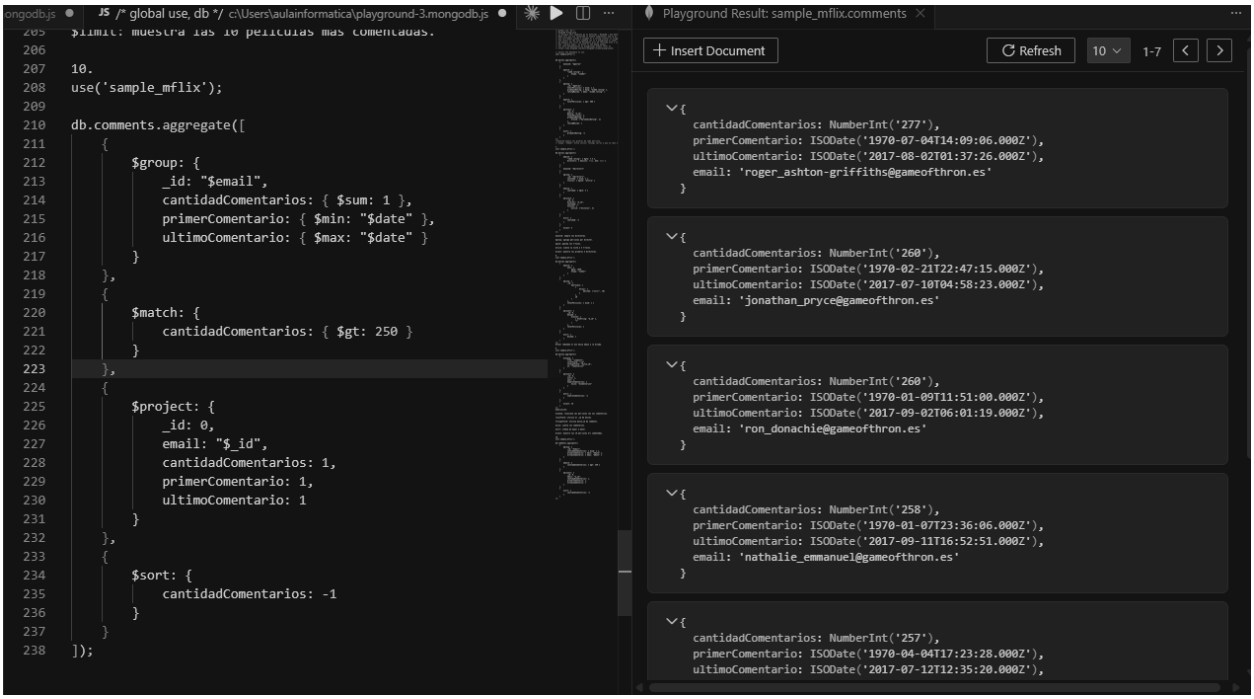
Punto 10 – Usuarios con más de 250 comentarios

Consulta:

```
db.comments.aggregate([
  { $group: { _id: "$email", cantidadComentarios: { $sum: 1 },
              primerComentario: { $min: "$date" }, ultimoComentario: { $max: "$date" } } },
  { $match: { cantidadComentarios: { $gt: 250 } } },
```

```
  { $project: { _id: 0, email: "$_id", cantidadComentarios: 1, primerComentario: 1, ultimoComentario: 1
} },
  { $sort: { cantidadComentarios: -1 } }
]);
```

Resultado:



`$group` por email cuenta los comentarios y obtiene la fecha del primero con `$min` y del último con `$max`; luego `$match` deja a los que pasan de 250. Son 7 usuarios, encabezados por roger_ashton-griffiths (277 comentarios, entre 1970 y 2017).

Parte C – Escritura e índices

Punto 11 – Colección peliculas_pareja con $out

Consulta:

```
db.movies.aggregate([
  { $match: { year: 2015, "imdb.rating": { $type: "number" } } },
  { $project: { _id: 0, titulo: "$title", generos: "$genres", rating: "$imdb.rating",
      categoria: { $switch: { branches: [
          { case: { $gte: ["$imdb.rating", 7.5] }, then: "Alta" },
          { case: { $gte: ["$imdb.rating", 6] },   then: "Media" } ],
        default: "Baja" } } } },
  { $out: "peliculas_pareja" }
]);
db.peliculas_pareja.countDocuments({});
```

Resultado:

```
========== PUNTO 11 ==========
Colección peliculas_pareja creada.
Total de películas: 432
Por categoría:
{
  _id: 'Alta',
  cantidad: 139
}
{
  _id: 'Baja',
  cantidad: 62
}
{
  _id: 'Media',
  cantidad: 231
}
Ejemplo de documentos:
{
  _id: ObjectId('6aea5b619ef0b9c7850fa80a'),
  titulo: 'Jurassic World',
  generos: [
    'Action',
    'Adventure',
    'Sci-Fi'
  ],
  rating: 7.3,
  categoria: 'Media'
}
{
  _id: ObjectId('6aea5b619ef0b9c7850fa80b'),
  titulo: 'The Stanford Prison Experiment',
  generos: [
    'Drama',
    'Thriller'
  ],
  rating: 7.2,
  categoria: 'Media'
}
{
  _id: ObjectId('6aea5b619ef0b9c7850fa80c'),
  titulo: 'Ex Machina',
  generos: [
    'Drama',
    'Mystery',
    'Romance'
  ],
```

El pipeline filtra las películas de 2015 con `$match` y proyecta título, géneros, rating y una `categoria` calculada con `$switch` (Alta ≥ 7,5; Media ≥ 6; Baja en otro caso). `$out` guarda el resultado en `peliculas_pareja`: **432 películas**, 139 Alta, 231 Media y 62 Baja. Se filtran solo los ratings numéricos (`$type: "number"`), porque los valores vacíos (`""`) se comparan como mayores que cualquier número y quedarían mal clasificados como Alta.

Punto 12 – updateMany sobre la categoría Media

Consulta:

```
const resultado12 = db.peliculas_pareja.updateMany(
  { categoria: "Media" },
  { $set: { revisado: true }, $inc: { rating: 0.1 } }
);
resultado12.matchedCount;    // 231
resultado12.modifiedCount;   // 231
```

Resultado:

```
========== PUNTO 12 ==========
matchedCount: 231
modifiedCount: 231
Ejemplo de películas actualizadas:
{
  _id: ObjectId('6aea5b619ef0b9c7850fa80a'),
  titulo: 'Jurassic World',
  generos: [
    'Action',
    'Adventure',
    'Sci-Fi'
  ],
  rating: 7.3999999999999995,
  categoria: 'Media',
  revisado: true
}
{
  _id: ObjectId('6aea5b619ef0b9c7850fa80b'),
  titulo: 'The Stanford Prison Experiment',
  generos: [
    'Drama',
    'Thriller'
  ],
  rating: 7.3,
  categoria: 'Media',
  revisado: true
}
{
  _id: ObjectId('6aea5b619ef0b9c7850fa80e'),
  titulo: 'The Wedding Ringer',
  generos: [
    'Comedy',
    'Romance'
  ],
  rating: 6.8,
  categoria: 'Media',
  revisado: true
}
{
  _id: ObjectId('6aea5b619ef0b9c7850fa812'),
  titulo: 'Sam',
  generos: [
    'Comedy',
    'Fantasy',
    'Romance'
  ],
  rating: 7,
```

`updateMany` agrega `revisado: true` con `$set` y suma 0,1 al rating con `$inc` en todas las películas Media. **matchedCount = 231** (documentos que cumplen el filtro) y **modifiedCount = 231** (documentos que cambiaron); coinciden porque ninguno tenía ya esos valores. Valores como 7,3999999999999995 son el error normal de los decimales en Double, el mismo que el manual advierte para el dinero.

Punto 13 – deleteMany sobre la categoría Baja

Consulta:

```
const resultado13 = db.peliculas_pareja.deleteMany({ categoria: "Baja" });
resultado13.deletedCount;    // 62
```

Resultado:

```
========== PUNTO 13 ==========
Películas Baja antes de borrar: 62
deletedCount: 62
Películas Baja restantes: 0

========== PUNTO 14: ANTES ==========
nReturned: 10
totalKeysExamined: 0
totalDocsExamined: 21349
executionTimeMillis: 21
Etapas del plan ganador: PROJECTION_DEFAULT <- SORT <- COLLSCAN

Índice ix_movies_genres_rating_year creado.

========== PUNTO 14: DESPUÉS ==========
nReturned: 10
totalKeysExamined: 20
totalDocsExamined: 10
executionTimeMillis: 1
Etapas del plan ganador: LIMIT <- PROJECTION_DEFAULT <- FETCH <- IXSCAN

Índice ix_movies_genres_rating_year eliminado.
PARTE C FINALIZADA.
true
Atlas atlas-hfi6hu-shard-0 [primary] sample_mflix>
```

`deleteMany con el filtro categoria = Baja` eliminó **62 documentos** (`deletedCount`), los mismos 62 que había antes de borrar. Un conteo posterior confirma que quedan 0 películas Baja.

Punto 14 – explain antes y después del índice ESR

Consulta:

```
const filtro1 = { genres: "Drama", year: { $gte: 2000, $lte: 2010 }, "imdb.rating": { $gte: 8 } };
const consulta = () => db.movies.find(filtro1, { _id: 0, title: 1, year: 1, "imdb.rating": 1 })
                                .sort({ "imdb.rating": -1 }).limit(10);
consulta().explain("executionStats");                       // ANTES: COLLSCAN
db.movies.createIndex({ genres: 1, "imdb.rating": -1, year: 1 },
                      { name: "ix_movies_genres_rating_year" });   // E - S - R
consulta().explain("executionStats");                       // DESPUÉS: IXSCAN
db.movies.dropIndex("ix_movies_genres_rating_year");
```

Resultado:

Se usó la consulta del punto 1 (Drama, año 2000–2010, rating ≥ 8, orden por rating descendente, 10 resultados) con el índice `genres + imdb.rating (descendente) + year`, que se eliminó al final.

Métrica	Sin índice	Con índice ESR
Etapa del plan ganador	SORT ← COLLSCAN	FETCH ← IXSCAN (sin SORT)
totalDocsExamined	21.349	10
totalKeysExamined	0	20
nReturned	10	10
executionTimeMillis	21	1

```
========== PUNTO 14: DESPUÉS ==========
nReturned: 10
totalKeysExamined: 20
totalDocsExamined: 10
executionTimeMillis: 1
Etapas del plan ganador: LIMIT <- PROJECTION_DEFAULT <- FETCH <- IXSCAN

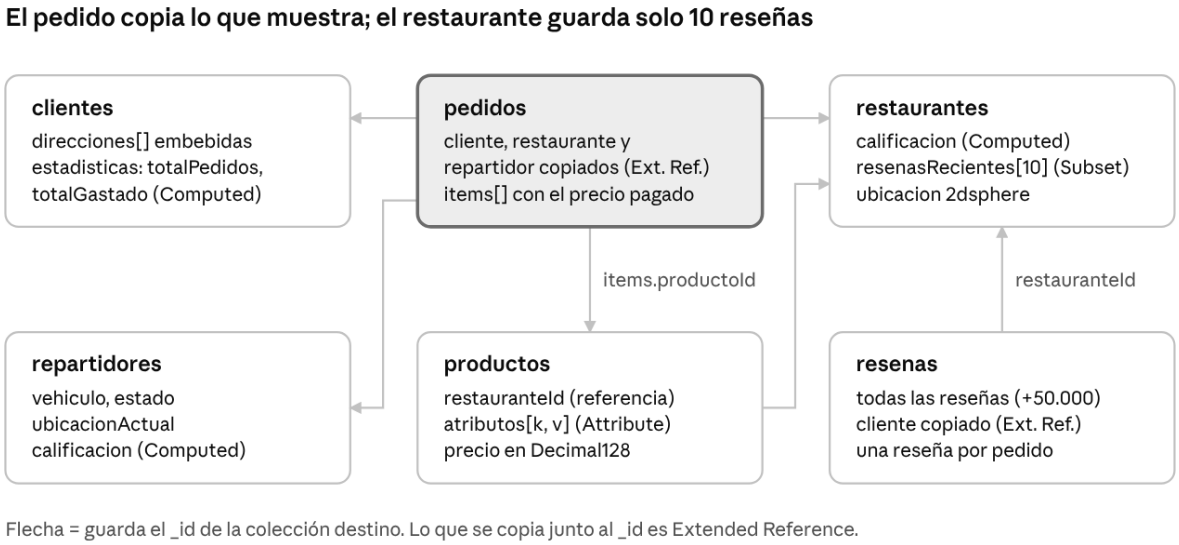
Índice ix_movies_genres_rating_year eliminado.
PARTE C FINALIZADA.
true
Atlas atlas-hfi6hu-shard-0 [primary] sample_mflix>
```

Orden del índice según ESR: `genres` es la igualdad (E), `imdb.rating` el campo de orden (S) y `year` el rango (R). Por eso desaparece la etapa SORT: el índice entrega las películas ya ordenadas por rating. Las 20 claves examinadas frente a 10 documentos leídos muestran que MongoDB descartó los años fuera de 2000–2010 usando solo el índice, sin leer esos documentos.

Parte D – Punto 15: Caso 1, plataforma de domicilios

El pedido guarda una copia del plato, el precio y los datos del restaurante del momento de la compra, y el restaurante guarda solo sus 10 reseñas más recientes con la calificación ya calculada. Así el historial de pedidos y la ficha del restaurante se responden leyendo una sola colección, sin `$lookup`.

Diagrama del modelo



Caso 1 · 6 colecciones, referencias y copias

Cada flecha sale de la colección que guarda la referencia. El modelo tiene 6 colecciones; las direcciones del cliente van embebidas porque son pocas y se leen siempre con él.

Patrones aplicados y qué pasaría sin ellos

Patrón	Dónde	Por qué se aplica	Si no se aplicara
Extended Reference	`pedidos.cliente`, `pedidos.restaurante`, `pedidos.repartidor`, `pedidos.items`, `resenas.cliente`	El historial muestra restaurante, plato y precio sin `$lookup`. Copiar los ítems es además una regla de negocio: el precio pagado no cambia si el menú cambia.	Con solo el `_id`, cada historial haría `$lookup` a productos y restaurantes y mostraría el precio de hoy, no el pagado.
Subset	`restaurantes.resenasRecientes` (10) y la colección `resenas` (todas)	La ficha del restaurante solo muestra las 10 más recientes; las demás se paginan bajo demanda.	Embebiendo 50.000 reseñas de unos 300 bytes, el documento llega a unos 15 MB, cerca del límite de 16 MB, y cada visita a la ficha movería megas.
Computed	`restaurantes.calificacion`, `clientes.estadisticas`,	La calificación se lee en cada visita y en cada ranking, y solo se escribe cuando llega una reseña.	El promedio exigiría un `$avg` sobre 50.000 reseñas por visita, y el top 5 recorrería todas las reseñas de la ciudad.

Patrón	Dónde	Por qué se aplica	Si no se aplicara
	`pedidos.total`		
Attribute	`productos.atributos` (pares k/v)	Cada plato tiene atributos distintos: tamaño, picante, vegano, calorías.	Un campo por atributo dejaría campos vacíos en casi todos los productos y un índice por atributo; con k/v basta un índice compuesto.
Embebido / Referencia	Direcciones embebidas en `clientes`; `productos` y `resenas` referenciados	Las direcciones son pocas y se leen con el cliente; el menú y las reseñas crecen y se consultan por separado.	Embeber menú y reseñas haría crecer el restaurante sin límite.

Índices propuestos

Colección	Índice	Campos	Para qué
pedidos	`ix_pedidos_cliente_fecha`	cliente._id, fecha (desc)	Consulta (a): igualdad por cliente y luego fecha (ESR)
pedidos	`ix_pedidos_restaurante_fecha`	restaurante._id, fecha (desc)	Pedidos del día en el panel del restaurante
restaurantes	`ix_restaurantes_ranking`	ciudad, activo, calificacion.promedio (desc), calificacion.total (desc)	Consulta (b): dos igualdades y luego el orden
restaurantes	`ix_restaurantes_ubicacion`	ubicacion (2dsphere)	Restaurantes cercanos al cliente
resenas	`ix_resenas_restaurante_fecha`	restauranteId, fecha (desc)	Paginar las reseñas que no están en el subset
resenas	`ix_resenas_pedido`	pedidoId (único)	Una reseña por pedido
productos	`ix_productos_menu`	restauranteId, categoria	Mostrar el menú por secciones
productos	`ix_productos_atributos`	atributos.k, atributos.v	Filtrar por cualquier atributo con un solo índice
clientes	`ix_clientes_email`	email (único)	Inicio de sesión

Consulta (a): pedidos de un cliente en el último mes con el total pagado

Consulta:

```
use("taller_parteD");
```

```
const hace30dias = new Date(Date.now() - 30 * 24 * 3600 * 1000);
db.pedidos.aggregate([
  { $match: { "cliente._id": ObjectId("6720a1b2c3d4e5f602000001"),
              fecha: { $gte: hace30dias }, estado: { $ne: "cancelado" } } },
  { $sort: { fecha: -1 } },
  { $facet: {
      pedidos: [ { $project: { _id: 0, codigo: 1, fecha: 1, estado: 1,
                               restaurante: "$restaurante.nombre", total: 1 } } ],
      resumen: [ { $group: { _id: null, cantidadPedidos: { $sum: 1 }, totalPagado: { $sum: "$total" } } },
                 { $project: { _id: 0 } } ] } }
]);
```

Resultado:

```
################ PUNTO 15 – CASO 1: DOMICILIOS ################

=== Caso 1 (a) Pedidos de Laura en los últimos 30 días ===
[
  {
    pedidos: [
      {
        codigo: 'PED-2026-000981',
        fecha: ISODate('2026-11-02T22:15:00.000Z'),
        estado: 'entregado',
        total: Decimal128('77100'),
        restaurante: 'La Fonda del Arriero'
      },
      {
        codigo: 'PED-2026-000944',
        fecha: ISODate('2026-10-28T01:30:00.000Z'),
        estado: 'entregado',
        total: Decimal128('48300'),
        restaurante: 'Wok Bambú'
      },
      {
        codigo: 'PED-2026-000902',
        fecha: ISODate('2026-10-20T18:05:00.000Z'),
        estado: 'entregado',
        total: Decimal128('41700'),
        restaurante: 'Arepas de la 85'
      }
    ],
    resumen: [
      {
        cantidadPedidos: 3,
        totalPagado: Decimal128('167100')
      }
    ]
  }
]

=== Caso 1 (b) Top 5 restaurantes de Bogotá ===
[
  {
    nombre: 'Arepas de la 85',
    localidad: 'Chapinero',
    calificacion: {
      promedio: 4.75,
      total: 12950
    }
  },
```

El pipeline filtra por cliente, últimos 30 días y estado distinto de cancelado, ordena por fecha y con `$facet` devuelve en una sola respuesta el listado y el resumen. Resultado: **3 pedidos y totalPagado de 167.100 COP**; el pedido cancelado y el de septiembre quedan fuera. No hay `$lookup` porque el nombre del restaurante ya viene copiado en el pedido (Extended Reference).

Consulta (b): top 5 restaurantes mejor calificados de Bogotá

Consulta:

```
db.restaurantes.find(
  { ciudad: "Bogotá", activo: true, "calificacion.total": { $gte: 50 } },
  { _id: 0, nombre: 1, localidad: 1, "calificacion.promedio": 1, "calificacion.total": 1 }
).sort({ "calificacion.promedio": -1, "calificacion.total": -1 }).limit(5);

// Consulta extra (Attribute): platos veganos con picante <= 2
db.productos.find({ atributos: { $all: [
  { $elemMatch: { k: "aptoVegano", v: true } },
  { $elemMatch: { k: "nivelPicante", v: { $lte: 2 } } } ] } }, { _id: 0, nombre: 1, precio: 1 });
```

Resultado:

```
  {
    nombre: 'Wok Bambú',
    localidad: 'Usaquén',
    calificacion: {
      promedio: 4.75,
      total: 8120
    }
  },
  {
    nombre: 'La Fonda del Arriero',
    localidad: 'Chapinero',
    calificacion: {
      promedio: 4.6,
      total: 52341
    }
  },
  {
    nombre: 'Pasta Nonna Lucía',
    localidad: 'Teusaquillo',
    calificacion: {
      promedio: 4.6,
      total: 2205
    }
  },
  {
    nombre: 'Parrilla El Llanero',
    localidad: 'Kennedy',
    calificacion: {
      promedio: 4.4,
      total: 3410
    }
  }
]

=== Caso 1 extra: veganos con picante hasta 2 ===
[
  {
    nombre: 'Pad thai de tofu',
    precio: Decimal128('32900')
  }
]
```

Es un `find` sobre `calificacion.promedio`, que ya está calculado (Computed), ordenado por promedio y luego por número de reseñas. Se exige un mínimo de 50 reseñas para que un restaurante con pocas no encabece el ranking: Burger Distrito (30 reseñas) queda fuera, y Bandeja Express no aparece porque es de Medellín. Arepas de la 85 supera a Wok Bambú con el mismo 4,75 porque tiene más reseñas.

La consulta extra muestra el patrón Attribute: platos veganos con picante hasta 2. Solo cumple el **Pad thai de tofu**; la limonada es vegana pero no tiene el atributo de picante.

Documentos de ejemplo

```
=== Caso 1 extra: veganos con picante hasta 2 ===
[
  {
    nombre: 'Pad thai de tofu',
    precio: Decimal128('32900')
  }
]

=== Caso 1: un documento de ejemplo por colección ===

--- restaurantes (8 documentos) ---
{
  _id: ObjectId('6720a1b2c3d4e5f601000001'),
  nombre: 'La Fonda del Arriero',
  ciudad: 'Bogotá',
  localidad: 'Chapinero',
  ubicacion: {
    type: 'Point',
    coordinates: [
      -74.0602,
      4.6486
    ]
  },
  categorias: [
    'colombiana',
    'parrilla'
  ],
  activo: true,
  calificacion: {
    promedio: 4.6,
    total: 52341,
    suma: 240769,
    distribucion: {
      '1': 600,
      '2': 900,
      '3': 2400,
      '4': 11036,
      '5': 37405
    }
  },
  resenasRecientes: [
    {
      resenaId: ObjectId('6720a1b2c3d4e5f606000003'),
      cliente: 'Sofía R.',
      estrellas: 5,
      comentario: 'La mejor bandeja de Chapinero',
      fecha: ISODate('2026-11-03T18:20:00.000Z')
```

`restaurantes`: la reseña nueva de Sofía actualizó en una sola operación el total (52.341), la suma, la distribución de 5 estrellas y la lista de reseñas recientes, donde quedó de primera.

```
--- resenas (3 documentos) ---
{
  _id: ObjectId('6720a1b2c3d4e5f606000001'),
  restauranteId: ObjectId('6720a1b2c3d4e5f601000001'),
  pedidoId: ObjectId('6720a1b2c3d4e5f605000001'),
  cliente: {
    _id: ObjectId('6720a1b2c3d4e5f602000001'),
    nombre: 'Laura G.'
  },
  estrellas: 5,
  comentario: 'Llegó caliente y rápido',
  fecha: ISODate('2026-11-02T23:10:00.000Z')
}

--- productos (6 documentos) ---
{
  _id: ObjectId('6720a1b2c3d4e5f604000001'),
  restauranteId: ObjectId('6720a1b2c3d4e5f601000001'),
  nombre: 'Bandeja paisa',
  categoria: 'platos fuertes',
  precio: Decimal128('38900'),
  disponible: true,
  adiciones: [
    {
      nombre: 'Huevo extra',
      precio: Decimal128('3500')
    },
    {
      nombre: 'Aguacate',
      precio: Decimal128('4000')
    }
  ],
  atributos: [
    {
      k: 'tamano',
      v: 'grande'
    },
    {
      k: 'nivelPicante',
      v: 0
    },
    {
      k: 'aptoVegano',
      v: false
    },
    {
```

`resenas` guarda una copia del nombre del cliente junto a su `_id`; `productos` guarda sus características como pares k/v en `atributos`.

```
--- clientes (2 documentos) ---
{
  _id: ObjectId('6720a1b2c3d4e5f602000001'),
  nombre: 'Laura Gómez',
  email: 'laura.gomez@example.com',
  telefono: '+57 300 000 0001',
  direcciones: [
    {
      alias: 'Casa',
      direccion: 'Cl 72 # 10-34 apto 501',
      localidad: 'Chapinero',
      principal: true
    },
    {
      alias: 'Universidad',
      direccion: 'Av Cra 9 # 131A-02',
      localidad: 'Usaquén',
      principal: false
    }
  ],
  estadisticas: {
    totalPedidos: 37,
    totalGastado: Decimal128('2148300'),
    ultimoPedido: ISODate('2026-11-02T22:15:00.000Z')
  }
}

--- repartidores (1 documentos) ---
{
  _id: ObjectId('6720a1b2c3d4e5f603000001'),
  nombre: 'Andrés Rojas',
  vehiculo: {
    tipo: 'moto',
    placa: 'ABC12D'
  },
  estado: 'disponible',
  ubicacionActual: {
    type: 'Point',
    coordinates: [
      -74.0571,
      4.6512
    ]
  },
  calificacion: {
    promedio: 4.8,
    total: 1210,
    suma: 5808
```

`clientes` tiene las direcciones embebidas y las estadísticas precalculadas; `repartidores` guarda su vehículo, su ubicación y su calificación.

```
--- pedidos (5 documentos) ---
{
  _id: ObjectId('6720a1b2c3d4e5f605000001'),
  codigo: 'PED-2026-000981',
  fecha: ISODate('2026-11-02T22:15:00.000Z'),
  estado: 'entregado',
  cliente: {
    _id: ObjectId('6720a1b2c3d4e5f602000001'),
    nombre: 'Laura Gómez',
    telefono: '+57 300 000 0001'
  },
  repartidor: {
    _id: ObjectId('6720a1b2c3d4e5f603000001'),
    nombre: 'Andrés Rojas',
    placa: 'ABC12D'
  },
  restaurante: {
    _id: ObjectId('6720a1b2c3d4e5f601000001'),
    nombre: 'La Fonda del Arriero',
    ciudad: 'Bogotá'
  },
  items: [
    {
      productoId: ObjectId('6720a1b2c3d4e5f604000001'),
      nombre: 'Bandeja paisa',
      precioUnitario: Decimal128('38900'),
      cantidad: 1,
      adiciones: [
        {
          nombre: 'Huevo extra',
          precio: Decimal128('3500')
        }
      ],
      subtotal: Decimal128('42400')
    },
    {
      productoId: ObjectId('6720a1b2c3d4e5f604000002'),
      nombre: 'Limonada de coco',
      precioUnitario: Decimal128('12900'),
      cantidad: 2,
      adiciones: [],
      subtotal: Decimal128('25800')
    }
  ],
  direccionEntrega: {
    direccion: 'Cl 72 # 10-34 apto 501',
```

`pedidos` copia los datos de cliente, repartidor y restaurante, y en cada ítem el nombre y el precio del plato al momento de la compra.

Índices y explain

```
      ],
      subtotal: Decimal128('42400')
    },
    {
      productoId: ObjectId('6720a1b2c3d4e5f604000002'),
      nombre: 'Limonada de coco',
      precioUnitario: Decimal128('12900'),
      cantidad: 2,
      adiciones: [],
      subtotal: Decimal128('25800')
    }
  ],
  direccionEntrega: {
    direccion: 'Cl 72 # 10-34 apto 501',
    localidad: 'Chapinero'
  },
  subtotal: Decimal128('68200'),
  costoEnvio: Decimal128('5900'),
  propina: Decimal128('3000'),
  total: Decimal128('77100'),
  metodoPago: 'tarjeta',
  historialEstados: [
    {
      estado: 'creado',
      fecha: ISODate('2026-11-02T21:40:00.000Z')
    },
    {
      estado: 'entregado',
      fecha: ISODate('2026-11-02T22:15:00.000Z')
    }
  ]
}

=== Caso 1: índices ===
restaurantes: _id_, ix_restaurantes_ranking, ix_restaurantes_ubicacion
resenas: _id_, ix_resenas_restaurante_fecha, ix_resenas_pedido
productos: _id_, ix_productos_menu, ix_productos_atributos
clientes: _id_, ix_clientes_email
pedidos: _id_, ix_pedidos_cliente_fecha, ix_pedidos_restaurante_fecha

=== Caso 1: explain de las consultas de negocio ===
  [explain (a) pedidos del cliente] plan: FETCH <- IXSCAN | docsExaminados: 4 | nReturned: 3
  [explain (b) top 5 restaurantes] plan: LIMIT <- FETCH <- IXSCAN | docsExaminados: 5 | nReturned: 5

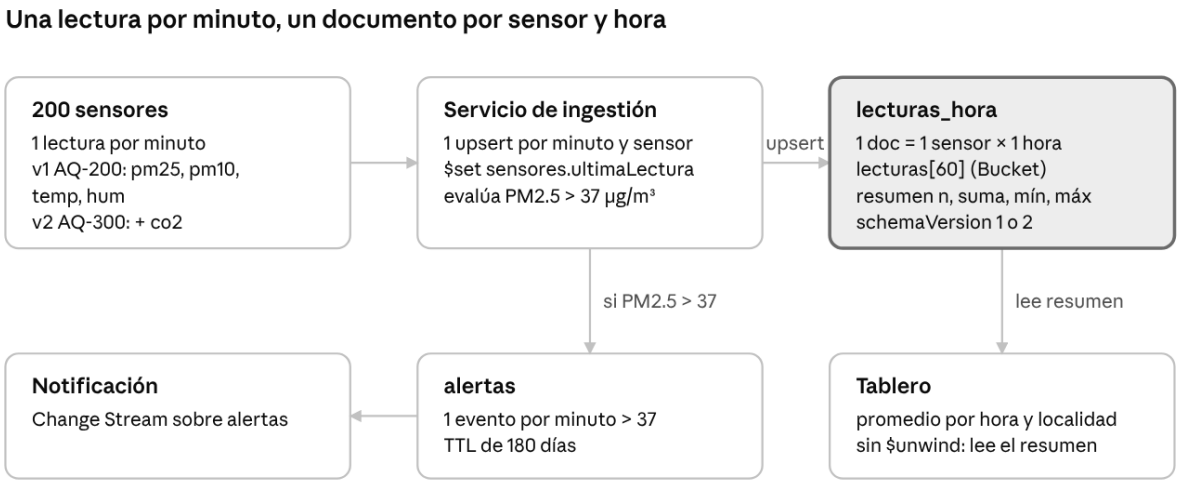
CASO 1 FINALIZADO.
true
Atlas atlas-hfi6hu-shard-0 [primary] taller_parteD>
```

Las dos consultas de negocio usan índice (`IXSCAN`). En (a) se examinan 4 documentos y se devuelven 3: el cuarto es el pedido cancelado, que se descarta después porque `estado` no está en el índice. En (b) se examinan y devuelven 5, porque el índice ya entrega los restaurantes ordenados.

Parte D – Punto 16: Caso 2, calidad del aire en Bogotá

Cada documento de `lecturas_hora` agrupa las 60 lecturas de un sensor en una hora y trae su resumen ya calculado. Las 288.000 lecturas diarias de la ciudad (200 sensores × 1.440 minutos) quedan en 4.800 documentos por día, 60 veces menos, y el tablero lee solo los resúmenes.

Diagrama del modelo



Caso 2 · de la lectura al bucket, la alerta y el tablero

El modelo tiene 3 colecciones: `sensores` (catálogo), `lecturas_hora` (los buckets) y `alertas`. Las dos últimas se relacionan con el sensor por `sensorId` y copian la `localidad` para agrupar sin `$lookup`.

Patrones aplicados y qué pasaría sin ellos

Patrón	Dónde	Por qué se aplica	Si no se aplicara
Bucket	`lecturas_hora`: 60 lecturas por documento	Las lecturas llegan en orden y se consultan por rango de tiempo. Cada bucket pesa unos 6 KB, muy lejos de los 16 MB.	Un documento por lectura: 288.000 al día y unos 105 millones al año, cada uno con su `_id` y sus entradas de índice. Con buckets son 4.800 al día y unos 1,75 millones al año.
Computed	`resumen` (n, suma, mín y máx por variable), `alertasPm25`, `sensores.ultimaLectura`	El tablero lee promedios horarios todo el tiempo. Se guardan suma y n para que el promedio de varios sensores sea exacto: suma de sumas sobre suma de n.	Cada consulta tendría que hacer `$unwind` de las 60 lecturas de cada bucket, y el estado actual de los sensores obligaría a ordenar millones de lecturas.
Schema Versioning	`schemaVersion` en `sensores` y	Los sensores nuevos (AQ-300) agregan	Habría que reescribir millones de buckets para

Patrón	Dónde	Por qué se aplica	Si no se aplicara
	`lecturas_hora`	CO_2. Sus documentos llevan la versión 2 y los históricos siguen en versión 1, sin migrar.	agregar `co2: null`, o la aplicación no podría distinguir “este sensor no mide CO_2” de “faltó la lectura”.
Extended Reference	`localidad` copiada en cada bucket y cada alerta	El tablero agrupa por localidad, y un sensor no cambia de localidad.	Cada consulta por localidad necesitaría `$lookup` a `sensores` sobre miles de buckets.

Índices propuestos

Colección	Índice	Campos	Para qué
lecturas_hora	`ix_lh_sensor_inicio`	sensorId, inicio (único)	El upsert de cada minuto encuentra su bucket y no se duplican
lecturas_hora	`ix_lh_localidad_inicio`	localidad, inicio	Consulta (a): igualdad por localidad y rango de horas
lecturas_hora	`ix_lh_alertas`	inicio (desc), parcial: solo horas con alertasPm25 > 0	Consulta (b): un índice pequeño con solo las horas con alerta
sensores	`ix_sensores_localidad` y `ix_sensores_ubicacion`	localidad; ubicacion (2dsphere)	Listar y ubicar los sensores en el mapa
alertas	`ix_alertas_ttl`	ts, TTL de 180 días	Borrado automático de las alertas viejas
alertas	`ix_alertas_sensor_ts`	sensorId, ts (desc)	Historial de alertas de un sensor

Carga de datos

Código de la carga (un minuto):

```
// Bucket + Computed: un upsert por lectura (en el script, 1.440 por sensor en un bulkWrite)
db.lecturas_hora.updateOne(
  { sensorId: "SEN-KEN-014", inicio: ISODate("2026-11-03T12:00:00Z") },
  { $push: { lecturas: l },
    $inc:  { nLecturas: 1, "resumen.pm25.n": 1, "resumen.pm25.suma": l.pm25,
             alertasPm25: l.pm25 > 37 ? 1 : 0 },
    $min:  { "resumen.pm25.min": l.pm25 },
    $max:  { "resumen.pm25.max": l.pm25 },
    $setOnInsert: { localidad: "Kennedy", schemaVersion: 1, fin: ISODate("2026-11-03T13:00:00Z") } },
  { upsert: true }
```

```
);
```

Resultado:

```
################ PUNTO 16 – CASO 2: CALIDAD DEL AIRE ################
SEN-KEN-014: 1440 lecturas, 127 minutos en alerta
SEN-KEN-022: 1440 lecturas, 17 minutos en alerta
SEN-CHA-003: 1440 lecturas, 0 minutos en alerta
Buckets creados: 72 (3 sensores x 24 horas = 72)

=== Caso 2 (a) Promedio horario de PM2.5 en Kennedy, 3 nov 2026 ===
[
  {
    maxPm25: 26.9,
    sensores: 2,
    hora: '00:00',
    promedioPm25: 20.7
  },
  {
    maxPm25: 26.8,
    sensores: 2,
    hora: '01:00',
    promedioPm25: 20.2
  },
  {
    maxPm25: 26.8,
    sensores: 2,
    hora: '02:00',
    promedioPm25: 20.3
  },
  {
    maxPm25: 27.4,
    sensores: 2,
    hora: '03:00',
    promedioPm25: 21.1
  },
  {
    maxPm25: 29.9,
    sensores: 2,
    hora: '04:00',
    promedioPm25: 22.7
  },
  {
    maxPm25: 34.2,
    sensores: 2,
    hora: '05:00',
    promedioPm25: 26.6
  },
  {
```

El script simula un día completo para 3 sensores (dos en Kennedy, uno en Chapinero): 4.320 lecturas, una por minuto, cada una guardada con un upsert. Resultado: **72 buckets** (3 sensores × 24 horas) y 144 minutos en alerta, todos en Kennedy.

Consulta (a): promedio horario de PM2.5 de una localidad en un día

Consulta:

```
db.lecturas_hora.aggregate([
  { $match: { localidad: "Kennedy",
              inicio: { $gte: ISODate("2026-11-03T05:00:00Z"), $lt: ISODate("2026-11-04T05:00:00Z") } }
},
  { $group: { _id: "$inicio", sumaPm25: { $sum: "$resumen.pm25.suma" },
              nPm25: { $sum: "$resumen.pm25.n" }, maxPm25: { $max: "$resumen.pm25.max" },
              sensores: { $sum: 1 } } },
  { $sort: { _id: 1 } },
  { $project: { _id: 0,
      hora: { $dateToString: { format: "%H:00", date: "$_id", timezone: "America/Bogota" } },
      promedioPm25: { $round: [{ $divide: ["$sumaPm25", "$nPm25"] }, 1] }, maxPm25: 1, sensores: 1 } }
]);
```

Resultado:

```
    promedioPm25: 22.7
  },
  {
    maxPm25: 34.2,
    sensores: 2,
    hora: '05:00',
    promedioPm25: 26.6
  },
  {
    maxPm25: 40.4,
    sensores: 2,
    hora: '06:00',
    promedioPm25: 33.3
  },
  {
    maxPm25: 44.5,
    sensores: 2,
    hora: '07:00',
    promedioPm25: 37.4
  },
  {
    maxPm25: 42.9,
    sensores: 2,
    hora: '08:00',
    promedioPm25: 34
  },
  {
    maxPm25: 35.1,
    sensores: 2,
    hora: '09:00',
    promedioPm25: 26.8
  },
  {
    maxPm25: 29,
    sensores: 2,
    hora: '10:00',
    promedioPm25: 22.1
  },
  {
    maxPm25: 27.5,
    sensores: 2,
    hora: '11:00',
    promedioPm25: 21
  },
  {
    maxPm25: 26.8,
    sensores: 2,
```


Se filtran los buckets de Kennedy del 3 de noviembre (hora de Bogotá), se agrupan por hora y el promedio se calcula como la suma de las sumas dividida por la suma de las n, no como promedio de promedios. El pico es a las **07:00, con 37,4 µg/m³ de promedio y 44,5 de máximo**, por encima del umbral de 37. No hace falta `$unwind`: se leen solo los resúmenes.

Consulta (b): sensores con alertas en la última semana

Consulta:

```
const hace7dias = new Date(Date.now() - 7 * 24 * 3600 * 1000);
db.lecturas_hora.aggregate([
  { $match: { inicio: { $gte: hace7dias }, alertasPm25: { $gt: 0 } } },
  { $group: { _id: "$sensorId", localidad: { $first: "$localidad" },
              minutosEnAlerta: { $sum: "$alertasPm25" }, horasConAlerta: { $sum: 1 },
              pm25Max: { $max: "$resumen.pm25.max" }, ultimaHora: { $max: "$inicio" } } },
  { $sort: { minutosEnAlerta: -1 } }
]);

// Consulta extra (Schema Versioning): CO2 solo en buckets versión 2
db.lecturas_hora.aggregate([
  { $match: { schemaVersion: { $gte: 2 }, localidad: "Chapinero" } },
  { $group: { _id: "$sensorId", sumaCo2: { $sum: "$resumen.co2.suma" }, nCo2: { $sum:
"$resumen.co2.n" } } },
  { $project: { co2Promedio: { $round: [{ $divide: ["$sumaCo2", "$nCo2"] }, 0] } } }
]);
```

Resultado:

```
=== Caso 2 (b) Sensores con alertas en los últimos 7 días ===
[
  {
    _id: 'SEN-KEN-014',
    localidad: 'Kennedy',
    minutosEnAlerta: 127,
    horasConAlerta: 6,
    pm25Max: 44.5,
    ultimaHora: ISODate('2026-11-04T00:00:00.000Z')
  },
  {
    _id: 'SEN-KEN-022',
    localidad: 'Kennedy',
    minutosEnAlerta: 17,
    horasConAlerta: 2,
    pm25Max: 39.8,
    ultimaHora: ISODate('2026-11-03T13:00:00.000Z')
  }
]

=== Caso 2 extra: CO2 promedio (solo sensores v2) ===
[
  {
    _id: 'SEN-CHA-003',
    co2Promedio: 427
  }
]

=== Caso 2: documentos de ejemplo ===

--- sensores v1 (AQ-200, sin CO2) ---
{
  _id: 'SEN-KEN-014',
  localidad: 'Kennedy',
  estacion: 'Parque Timiza',
  ubicacion: {
    type: 'Point',
    coordinates: [
      -74.1576,
      4.6097
    ]
  },
  modelo: 'AQ-200',
  schemaVersion: 1,
  variables: [
    'pm25',
```

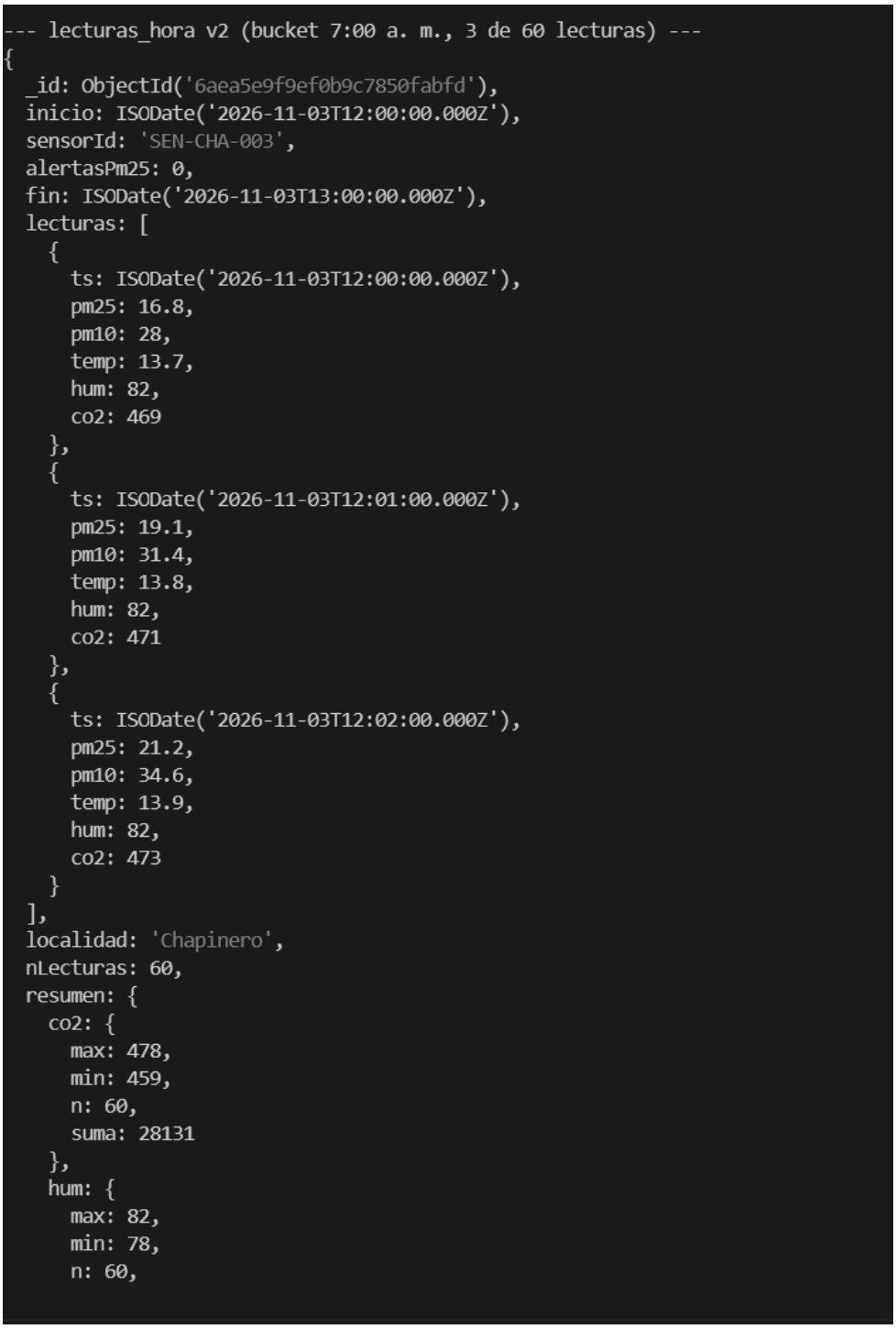
Se buscan los buckets de los últimos 7 días con `alertasPm25` mayor que 0 y se agrupan por sensor. **SEN-KEN-014** tuvo 127 minutos en alerta repartidos en 6 horas (máximo 44,5) y **SEN-KEN-022** 17 minutos en 2 horas (máximo 39,8). El sensor de Chapinero no aparece porque nunca pasó el umbral.

La consulta extra muestra el Schema Versioning: el CO_2 promedio (427 ppm) solo se calcula para los buckets de versión 2, es decir, para SEN-CHA-003.

Documentos de ejemplo

```
--- lecturas_hora v1 (bucket 7:00 a. m., 3 de 60 lecturas) ---
{
  _id: ObjectId('6aea5e979ef0b9c7850fabcc'),
  inicio: ISODate('2026-11-03T12:00:00.000Z'),
  sensorId: 'SEN-KEN-014',
  alertasPm25: 52,
  fin: ISODate('2026-11-03T13:00:00.000Z'),
  lecturas: [
    {
      ts: ISODate('2026-11-03T12:00:00.000Z'),
      pm25: 38.1,
      pm10: 60.9,
      temp: 13.8,
      hum: 82
    },
    {
      ts: ISODate('2026-11-03T12:01:00.000Z'),
      pm25: 40.5,
      pm10: 64.6,
      temp: 13.9,
      hum: 81
    },
    {
      ts: ISODate('2026-11-03T12:02:00.000Z'),
      pm25: 42.3,
      pm10: 67.2,
      temp: 13.9,
      hum: 81
    }
  ],
  localidad: 'Kennedy',
  nLecturas: 60,
  resumen: {
    hum: {
      max: 82,
      min: 77,
      n: 60,
      suma: 4767
    },
    pm10: {
      max: 69,
      min: 52,
      n: 60,
      suma: 3679.2999999999997
    },
    pm25: {
```

Bucket versión 1 (sensor de Kennedy, 7:00 a. m.): 60 lecturas sin CO_2, el resumen por variable y 52 de sus 60 minutos con PM2.5 por encima de 37. Se muestran 3 de las 60 lecturas.



```
--- lecturas_hora v2 (bucket 7:00 a. m., 3 de 60 lecturas) ---
{
  _id: ObjectId('6aea5e9f9ef0b9c7850fabfd'),
  inicio: ISODate('2026-11-03T12:00:00.000Z'),
  sensorId: 'SEN-CHA-003',
  alertasPm25: 0,
  fin: ISODate('2026-11-03T13:00:00.000Z'),
  lecturas: [
    {
      ts: ISODate('2026-11-03T12:00:00.000Z'),
      pm25: 16.8,
      pm10: 28,
      temp: 13.7,
      hum: 82,
      co2: 469
    },
    {
      ts: ISODate('2026-11-03T12:01:00.000Z'),
      pm25: 19.1,
      pm10: 31.4,
      temp: 13.8,
      hum: 82,
      co2: 471
    },
    {
      ts: ISODate('2026-11-03T12:02:00.000Z'),
      pm25: 21.2,
      pm10: 34.6,
      temp: 13.9,
      hum: 82,
      co2: 473
    }
  ],
  localidad: 'Chapinero',
  nLecturas: 60,
  resumen: {
    co2: {
      max: 478,
      min: 459,
      n: 60,
      suma: 28131
    },
    hum: {
      max: 82,
      min: 78,
      n: 60,
```

Bucket versión 2 (sensor de Chapinero, misma hora): las lecturas y el resumen incluyen `co2`, y `alertasPm25` es 0.

Alertas, índices y explain

```
    },
    pm10: {
      max: 37.9,
      min: 20.5,
      n: 60,
      suma: 1770.3000000000002
    },
    pm25: {
      max: 24.5,
      min: 15.2,
      n: 60,
      suma: 1199.4000000000003
    },
    temp: {
      max: 14.2,
      min: 13.5,
      n: 60,
      suma: 830.6000000000003
    }
  },
  schemaVersion: 2
}

--- alertas ---
{
  _id: ObjectId('6aea5e9890241498637e0bcb'),
  sensorId: 'SEN-KEN-014',
  localidad: 'Kennedy',
  variable: 'pm25',
  valor: 38.3,
  umbral: 37,
  ts: ISODate('2026-11-03T11:17:00.000Z'),
  estado: 'abierta'
}

=== Caso 2: índices ===
sensores: _id_, ix_sensores_localidad, ix_sensores_ubicacion
lecturas_hora: _id_, ix_lh_sensor_inicio, ix_lh_localidad_inicio, ix_lh_alertas
alertas: _id_, ix_alertas_ttl, ix_alertas_sensor_ts

=== Caso 2: explain de las consultas de negocio ===
  [explain (a) promedio horario Kennedy] plan: FETCH <- IXSCAN | docsExaminados: 48 | nReturned: 48
  [explain (b) alertas última semana] plan: FETCH <- IXSCAN | docsExaminados: 8 | nReturned: 8

CASO 2 FINALIZADO.
true
Atlas atlas-hfi6hu-shard-0 [primary] taller_parteD>
```

El bucket v2 cierra con `schemaVersion: 2`. La alerta de ejemplo registra una lectura de 38,3 contra el umbral de 37. Las dos consultas usan `IXSCAN`: en (a) se examinan 48 buckets y se devuelven 48 (2 sensores de Kennedy × 24 horas), y en (b) 8 y 8, las horas con alerta, porque el índice parcial solo contiene esas horas y no recorre las otras 64.